

长 沙 学 院

课程设计指导书

课 程 名 称 数据结构与算法课程设计

学 院 计算机科学与工程学院

专 业 软件工程

班 级 25 级软件 1-4 班

2026 年 6 月 17 日

课程名称：数据结构课程设计

课程编号：JX031117

主 笔 人：张成

主 审 人：何可可

一、课程设计的目的

《数据结构与算法课程设计》是软件工程专业在完成《数据结构与算法》理论教学后开设的一门重要实践课程。本课程设计旨在：

- 1. 加深理论理解：通过实际项目的开发，加深对线性表、树、图等基本数据结构及排序、查找等经典算法的理解，将抽象的理论知识转化为具体的工程实践能力。
- 2. 培养工程能力：初步掌握软件开发过程中的问题分析、系统设计、编码实现、测试验证等基本方法和技能，体验从需求到交付的完整项目流程。
- 3. 训练系统思维：培养针对实际问题进行数据结构选型、算法设计与性能分析的系统思维能力，理解“权衡与取舍”的工程哲学。
- 4. 提升综合素养：通过项目报告撰写和答辩陈述，提升技术文档写作能力、口头表达能力和学术诚信意识。

二、课程设计的总体要求

2.1 基本要求

- 1. 独立完成：课程设计必须由学生本人独立完成，严禁抄袭、代做等学术不端行为。
- 2. 规范开发：遵循模块化设计原则，按照“需求分析→方案设计→编码实现→测试验证→总结答辩”的流程推进项目。
- 3. 规范文档：按照规定的格式撰写课程设计报告，内容完整、逻辑清晰、术语准确。
- 4. 按时提交：严格按照进度安排完成各阶段任务，按时提交所有成果物。

2.2 时间管理建议

课程设计为期 2 周（10 个工作日），建议按以下节奏推进：

表 1 课程设计进度安排

阶段	时间	重点任务
需求分析与方案设计	第 1-2 天	理解需求、确定数据结构方案、完成逻辑结构图与 ADT 定义
编码实现	第 3-6 天	分模块编码、单元测试、逐步集成

测试与优化	第 7-8 天	系统测试、性能对比、复杂度验证
-------	---------	-----------------

三、课程设计与任务

详见任务书

四、需求分析与方案设计方法

4.1 如何读懂任务

拿到任务书后，按以下步骤理解需求：

1. 通读全文：先整体了解项目背景、功能需求和评分标准，建立全局认识。
2. 逐条拆解：将每个功能需求分解为具体的子任务，明确每个子任务的输入、处理和输出。
3. 标注优先级：区分核心功能（必须完成）和附加功能（冲击优秀），合理安排开发顺序。
4. 明确数据规范：仔细阅读数据字段定义，理解每个字段的类型、格式和业务含义。

4.2 如何进行数据结构抽象

数据结构抽象是从问题描述到数据结构定义的映射过程，是课程设计的第一个关键环节。建议按以下步骤进行：

步骤 1：识别数据实体

从问题描述中找出核心数据对象。例如在“校园选课记录检索与大数据分析系统”中，核心数据实体是“选课记录”，每条记录包含学号、姓名、课程名称、成绩等属性。

步骤 2：分析数据关系

分析数据实体之间的逻辑关系：

- 线性关系：数据元素按顺序排列（如按学号排列的学生列表）
- 树形关系：数据元素存在层次关系（如学院→专业→学生）
- 图形关系：数据元素存在多对多关联（如课程之间的先修关系）
- 散列关系：通过关键码直接访问（如按学号查找学生）

步骤 3：定义 ADT（抽象数据类型）

ADT 定义应包含三个部分：

- 数据对象：定义数据元素的构成（即结构体定义）

- 数据关系：定义数据元素之间的逻辑关系
- 基本操作：定义对该数据结构可执行的操作集合（初始化、插入、删除、查找、遍历等）

步骤 4：绘制逻辑结构图

用图形化的方式表达数据之间的逻辑关系：线性结构用箭头链条表示；树形结构用树状图表示；图形结构用节点+连线表示；散列结构用“键→值”映射图表示。

示例：选课记录系统的逻辑结构图应展示：

选课记录作为基本数据节点；节点之间的组织方式（按学号排序？按课程组织？按哈希分布？）；不同存储结构对应的不同组织形式。

4.3 如何进行方案对比与选型

方案对比是课程设计的核心难点，也是体现“分析辨别能力”（课程目标 2）的关键环节。对比维度：

表 4 方案对比维度

维度	考察内容	对比方法
时间复杂度	各操作的理论时间复杂度	查阅教材或文献，列出插入/查找/删除/遍历的复杂度
空间复杂度	存储相同数据所需的内存空间	分析各结构的内存开销（指针域、平衡因子等）
实现复杂度	编码实现的难易程度	评估代码量、调试难度、出错概率
适用场景	各结构适合什么类型的操作	结合具体操作频率分析

选型论证的写作结构：

1. 列出候选方案：明确你对比了哪几种数据结构
2. 给出对比依据：从上述维度逐一对比，最好用表格呈现
3. 明确选型结论：结合项目的具体约束（数据规模、操作频率等），给出最终选择
4. 提供量化理由：不仅仅是“我选了 X”，而是“因为 X 在查找操作上时间复杂度为 $O(\log n)$ ，而 Y 为 $O(n)$ ，而本项目查找操作最为频繁，因此选择 X”

常见错误：

- 只对比不结论，或结论缺乏依据
- 对比维度单一（只比时间复杂度，忽略空间和实现复杂度）
- 选型理由过于主观（“我觉得 X 更好用”而非“X 在……方面更优”）

五、编码实现规范

5.1 模块化设计原则

将系统划分为若干个功能独立的模块，每个模块职责单一、接口明确。模块划分示例如下：



图 1 模块划分示例

组队提示：两人可分别负责不同数据结构模块的实现（如一人负责 list.c，一人负责 avl.c），各自完成单元测试后再集成。

5.2 编码规范

命名规范：

- 结构体类型名使用大驼峰，如 StudentRecord、AVLNode
- 函数名使用小写下划线分隔，如 insert_record()、find_by_id()
- 常量使用全大写，如 MAX_NAME_LEN

注释规范：

- 每个文件头部注明文件名、作者、功能简述
- 每个函数前注明功能、参数说明、返回值说明
- 关键逻辑处添加行内注释，解释“为什么这样做”而非“做了什么”

示例：

```

/**
 * @brief 按学号查找选课记录
 * @param tree AVL树根节点指针
 * @param id 学号 (12位字符串)
 * @return 找到的记录指针, 未找到返回NULL
 */
Record* find_by_id(AVLNode* tree, const char* id) {
    // 在AVL树中查找: 利用二叉搜索树性质, 比较学号大小
    // 时间复杂度  $O(\log n)$ 
    while (tree) {
        int cmp = strcmp(id, tree->data.id);
        if (cmp == 0) return &tree->data;
        else if (cmp < 0) tree = tree->left;
        else tree = tree->right;
    }
    return NULL;
}

```

图 2 代码注释示例

5.3 内存管理

C 语言中内存管理是常见的问题来源, 需特别注意:

- 动态分配后及时释放: 每次 malloc/calloc 都应有对应的 free
- 避免内存泄漏: 程序退出前释放所有动态分配的内存
- 避免野指针: 指针使用前检查是否为 NULL, 释放后置为 NULL
- 可使用工具辅助检测: 如 Valgrind (Linux)、Dr.Memory (Windows)

5.4 调试建议

1. 分模块调试: 每完成一个模块, 立即测试其功能, 不要等全部写完再调试
2. 使用打印调试: 在关键位置添加 printf 输出中间状态
3. 边界测试: 特别关注空结构、单元素、满容量等边界情况
4. 版本控制: 建议使用 Git 进行版本管理, 每次重大修改前提交

六、测试方法与测试用例设计

6.1 测试的基本原则

测试的目的不是证明程序“能跑”，而是发现程序中可能存在的问题。一个好的测试用例应该能够暴露潜在的缺陷。

6.2 测试用例的分类

表 5 测试用例分类示例

类型	说明	示例（选课记录系统）
正常测试	验证功能在正常输入下是否正确	插入一条完整合法的选课记录，查询返回正确结果
边界测试	验证功能在极端输入下是否健壮	查找不存在的学号、删除空结构中的记录、插入第 10001 条记录
异常测试	验证功能在非法输入下是否优雅处理	输入非数字成绩、超长字符串、空文件加载

6.3 测试用例的设计方法

1. 等价类划分：将输入数据划分为若干等价类，从每个类中选取代表性数据进行测试。
例如成绩字段：

- 有效等价类：0-100 的整数
- 无效等价类：负数、大于 100 的数、非数字字符

2. 边界值分析：重点关注边界值。例如：

- 成绩：0、1、99、100、-1、101
- 数据规模：0 条、1 条、最大容量、最大容量+1

6.4 测试报告的内容

测试报告应包含：

1. 测试环境：操作系统、编译器版本、硬件配置
2. 测试用例列表：每个用例的编号、输入、预期输出、实际输出、是否通过
3. 缺陷分析：未通过的用例，分析原因及修复情况
4. 结论：测试覆盖率、通过率、系统质量评估

七、性能测试与复杂度验证方法

7.1 性能测试的准备工作

1. 准备多规模数据集：按任务要求准备不同规模的数据（100、1000、10000 条等）

2. 统一测试环境：在同一台机器、相同系统负载下进行测试
3. 多次测试取平均：每个操作至少运行 3 次，取平均耗时，减少偶然误差
4. 记录测试条件：记录 CPU 型号、内存大小、编译器版本等，便于结果复现

7.2 性能测试的执行

测试流程：1. 生成指定规模的数据集；2. 将数据加载到各数据结构中；3. 分别执行各操作（插入、查找、删除、遍历、排序），记录耗时；4. 记录内存占用；5. 更换数据规模，重复上述步骤。

计时方法：使用 `clock()` 函数（C 标准库）；或使用 `gettimeofday()`（Linux）获取微秒级精度；或使用 `QueryPerformanceCounter()`（Windows）获取高精度计时。

7.3 复杂度验证

复杂度验证的目的是将理论分析与实测数据对照，理解“理论”与“实际”的差距。
理论复杂度参考：

表 6 理论复杂度参考

操作	双向链表	AVL 树	哈希表
插入	$O(1)$ （头插）或 $O(n)$ （尾插需遍历）	$O(\log n)$	$O(1)$ （平均）
查找	$O(n)$	$O(\log n)$	$O(1)$ （平均）
删除	$O(n)$ （需先查找）	$O(\log n)$	$O(1)$ （平均）
遍历	$O(n)$	$O(n)$	$O(n)$ （但顺序不确定）
排序	$O(n \log n) \sim O(n^2)$	$O(n)$ （中序遍历即有序）	$O(n \log n)$ （需额外排序）

偏差分析：

实际运行耗时与理论预期可能存在偏差，常见原因包括：

1. 常数因子：理论复杂度忽略常数，但实际中常数因子影响显著（如哈希函数的计算开销）；
2. 缓存效应：顺序访问（数组/链表）的缓存命中率高于随机访问（树/哈希表）；
3. 递归开销：AVL 树的递归操作有函数调用开销；
4. 实现细节：不同的实现方式（如递归 vs 迭代）影响实际性能。

分析框架：

1. 列出各操作的理论时间复杂度；
2. 展示实测数据（表格形式）；

3. 对比理论与实际：哪些操作符合预期？哪些存在偏差？；
4. 分析偏差原因：结合上述因素逐一解释；
5. 得出结论：在什么条件下理论分析有效？什么条件下需要结合实测？

八、课程设计报告撰写规范

8.1 报告结构

课程设计报告应包含以下章节：

表 7 课程设计报告结构示例

章节	内容要求
封面	课程名称、题目、姓名、学号、指导教师、日期
第 1 章 问题分析	需求分析、功能范围、数据规模、约束条件
第 2 章 数据结构设计	逻辑结构图、ADT 定义、存储结构方案及选型理由
第 3 章 方案对比与选型论证	多方案对比（表格）、选型结论及量化理由、复杂度分析
第 4 章 实现说明	模块划分图、核心函数说明、关键代码片段（带注释）
第 5 章 测试与验证	测试用例设计、测试结果、复杂度验证（理论与实际对比）
第 6 章 总结	项目总结、收获与反思、可改进之处
参考文献	引用教材、文献、网络资源
附录	完整源代码（或代码仓库链接）

8.2 格式规范

课程设计报告应符合规范，如下所示（详见说明书模板）：

表 8 课程设计报告规范

项目	要求
页面设置	A4 纸，上下左右边距 2.5cm
正文字体	宋体小四号，1.5 倍行距
标题字体	黑体，一级标题三号，二级标题四号
图表	图表需有编号和标题，图中文字清晰可读
代码	使用等宽字体（如 Courier New），有行号或缩进
页码	正文从第 1 页开始编号，封面不编号

8.3 图表规范

逻辑结构图：使用 Visio、Draw.io、PowerPoint 或手绘后扫描；图中元素需有标注，关系清晰；图标题置于图下方，格式为“图 7.3-1 XXXXX”。

表格：表格需有表头，数据对齐；表标题置于表上方，格式为“表 7-3-2 XXXXX”。

代码片段：选择最能体现核心设计的关键代码（非全部代码）；需有行号和必要的注

释；代码前应有文字说明，解释这段代码的作用。

九、课程设计考核方法与成绩评定

9.1 考核方式

本课程采用“项目文档 + 项目代码 + 演示答辩”相结合的模式，重点考核学生的抽象建模能力、分析辨别能力、工程实现能力、实验验证能力与沟通表达能力。三个环节均为百分制，按权重加权计算总评成绩。

9.2 成绩构成

$$\text{总评成绩} = \text{项目文档} \times 40\% + \text{项目代码} \times 20\% + \text{演示答辩} \times 40\%$$

总评等级依据总评成绩，按以下规则转换为五级制：优秀（90-100）、良好（80-89）、中等（70-79）、及格（60-69）和不及格（60 分以下），严格按上述标准进行综合给分。

一票否决规则：若项目代码原始得分<60 分或演示答辩原始得分<60 分，则总评成绩直接判定为不及格。

9.3 评分标准

各评分项依据教学大纲中对应课程目标的 Rubric 进行等级评定给分。

十、注意事项

1. 课程设计必须**独立完成**。引用他人代码或思想需在报告中明确标注引用来源。如发现抄袭或代做，按学校相关规定处理如发现抄袭或代做，或组队中某成员未承担实质工作，按学校相关规定处理。
2. 任务书下发后，学生不能无故更换课题
3. 没有通过上机检查的学生其成绩直接记录不及格
4. 答辩结束后给出最终成绩